FLEX Disk Format Specification

FLEX Disk Format Specification

A complete description of the on-disk format used by TSC FLEX for the 6809, compiled from the *6809 FLEX Adaptation Guide* and the *FLEX Advanced Programmer's Guide*. All offsets are decimal unless written with `$` (hex). Byte 0 is the first byte of the structure being described.

> **Scope**
>
> Everything here comes from the two TSC guides, except the internal layout of the File Sector Map (§9), which those guides don't document. That section is marked accordingly.

---

1. Physical Format

Property	Value
Sector size	256 bytes
Track numbering	Starts at 0 (outermost track)
Sector numbering	Starts at 1
Encoding	Soft-sectored, essentially IBM-compatible except for the 256-byte sectors
Track 0 on double-density disks	Always single density (allows automatic density detection)
Side 0	Bottom of the disk (opposite the label)
Fill value at format time	TSC uses `$00` (IBM uses `$E5`); not critical

Disk addresses everywhere in FLEX are two bytes: **track**, then **sector**. A link of `$00 $00` means "no next sector."

On double-sided disks, the sector count in the table below is the total per track across both sides. Sectors are numbered continuously, side 0 first: on an 8" DS/SD disk, sectors 1–15 are side 0 and 16–30 are side 1.

1.1 Standard Distribution Formats

TSC defines two interchange formats that any FLEX system with the same drive size should be able to read:

Format	Tracks	Sectors/track
8" SS/SD	77 (0–76)	15 (1–15)
5¼" SS/SD	35 (0–34)	10 (1–10)

1.2 Other Defined Formats

These are TSC guidelines rather than fixed standards.

Disk type	Tracks	Sectors/track, tracks 1+ (one side / total)	Sectors on track 0 (one side / total)
8" DS/SD	77	15 / 30	15 / 30
8" DS/DD	77	26 / 52	15 / 30
8" SS/DD	77	26 / 26	15 / 15
5¼" SS/SD	40	10 / 10	10 / 10
5¼" DS/SD	35 or 40	10 / 20	(not given in source)

1.3 Sector Interleave

FLEX disks are interleaved: logical sectors are not physically adjacent, so the CPU has time to process one sector before the next passes under the head. The interleave affects speed only, not the logical format. FLEX always addresses sectors by their logical number (the ID written in the sector header). TSC's standard schemes are optimized for reading text files:

Physical	8" logical	5¼" logical
1	1	1
2	6	3
3	11	5
4	3	7
5	8	9
6	13	2
7	5	4

Physical	8" logical	5¼" logical
8	10	6
9	15	8
10	2	10
11	7	
12	12	
13	4	
14	9	
15	14	

8" is a 5:1 interleave and 5¼" is 2:1.

---

2. Disk Layout Overview

```
Track 0 (reserved for the system; never used for file data)
  Sector 1      Boot loader
  Sector 2      Boot loader (continued, if needed)
  Sector 3      System Information Record (SIR)
  Sector 4      Not described in the guides (reserved)
  Sector 5..N   Directory (a linked chain; can grow onto other tracks)

Tracks 1..last
  Data sectors: either in a file's chain or in the free chain
```

Every sector except the boot sectors uses the common sector format in §3.

---

3. Common Sector Format

Offset	Size	Contents
0	1	Track of the next sector in the chain
1	1	Sector of the next sector in the chain
2–3	2	Logical record number of this sector within its file (16-bit, big-endian)
4–255	252	Data

So each sector holds **252 bytes** of payload. The last sector of any chain has a link of `$00 $00`.

When a sector moves from the free chain into a file, its link is rewritten to point to the next sector of the file. The FMS maintains links; user programs never touch them.

---

4. Boot Sectors (Track 0, Sectors 1–2)

These hold the bootstrap loader that the ROM monitor (for example, SBUG's `D` command) loads, which in turn loads FLEX. They are raw code and do **not** follow the link/record header format. NEWDISK conventionally writes the loader to sector 1 from memory at `$C100`. A loader larger than 256 bytes also uses sector 2.

---

5. System Information Record (SIR) — Track 0, Sector 3

Offset	Size	Contents
0–1	2	`$00 $00` (cleared forward link)
2–15	14	Not defined
16–26	11	Volume name, ASCII
27–28	2	Volume number, binary
29–30	2	First free sector (track, sector): head of the free chain
31–32	2	Last free sector (track, sector): tail of the free chain
33–34	2	Number of free data sectors, binary
35–37	3	Creation date: month, day, year (binary; year is two digits)
38	1	Highest track number on the disk
39	1	Highest sector number on a track

Notes:

- On a freshly formatted disk with no bad sectors, the first free sector is track 1 sector 1, and the last free sector is the last sector of the last track.
- The data sector count excludes all of track 0 and is decremented for every bad sector NEWDISK links out. FLEX uses it to know when the disk is full.
- The highest track is the actual last track number (76, `$4C`, for a 77-track 8" disk). The highest sector is the total per track across both sides on double-sided disks.

> ⚠ **OCR fix**
>
> In the pdftotext conversion of the Adaptation Guide, the SIR rows for offsets 33–38 came out scrambled. The table above is the corrected layout.

---

6. Directory

6.1 Directory Sectors

The directory is a linked chain starting at **track 0, sector 5**. Initially it holds every sector from 5 to the end of track 0, but it can extend onto other tracks when full.

Offset	Size	Contents
0–1	2	Link to next directory sector (track, sector)
2–3	2	Record number bytes (part of the common header)
4–15	12	Unused
16–255	240	10 directory entries × 24 bytes

Directory capacity of a fresh disk is (sectors on track 0 − 4) × 10 entries. That's 110 for 8" SS/SD and 60 for 5¼" SS/SD.

6.2 Directory Entry (24 bytes)

This is the same structure as FCB bytes 4–27. Offsets below are relative to the start of the entry, with the FCB offset alongside.

Entry offset	FCB offset	Size	Contents
0–7	4–11	8	File name, ASCII, left-justified, zero-padded
8–10	12–14	3	Extension, ASCII, left-justified, zero-padded
11	15	1	Attributes (see below)
12	16	1	Reserved
13–14	17–18	2	First sector of file (track, sector)
15–16	19–20	2	Last sector of file (track, sector)
17–18	21–22	2	File size in sectors (16-bit)

Entry offset	FCB offset	Size	Contents
19	23	1	File Sector Map flag: non-zero (usually `$02`) means a random file
20	24	1	Reserved
21–23	25–27	3	Creation date: month, day, year (two digits)

Entry status, from the first byte of the name:

- `$00`: never used. The scan of the directory can stop at the first such entry.
- High bit set (byte is negative): deleted.
- Anything else: an active file.

Name rules: the name and extension must each start with a letter and contain only letters, digits, hyphens and underscores. Files with null extensions should not be created.

Attribute byte: only the upper four bits are defined. Undefined bits must be 0.

Bit	Meaning when set
7	Write protect
6	Delete protect
5	Read protect
4	Catalog protect (hidden from `CAT`)
3–0	Undefined, must be 0

---

7. Free Chain

All sectors not belonging to a file form a single linked chain, recorded in the SIR by its first and last sector. After NEWDISK, the chain runs from track 1 sector 1 through the last sector of the last track, in logical order, with bad sectors linked out. All free sectors have record number bytes of 0 and zeroed data.

Allocation takes sectors from the head of the free chain. Deleting a file appends its chain to the tail. Over time this fragments the disk, which only matters for random files (see §9).

---

8. File Data

A file is a chain of sectors linked through bytes 0–1, starting at the directory entry's first sector and ending at its last sector, which has a link of `$00 $00`. Bytes 2–3 of each sector hold its logical record number within the file, starting at 1 for the first data sector (record 0 is the FSM in random files).

FLEX doesn't distinguish file types at the disk level. Binary and text are conventions about the 252-byte data payloads, treated as one continuous byte stream across the chain.

8.1 Binary Files

The byte stream is a sequence of records. Records can span sector boundaries, and a sector can hold several.

Data record

Offset	Size	Contents
0	1	`$02` (STX), start of record
1–2	2	Load address, big-endian
3	1	Byte count *n*
4..3+*n*	*n*	Data

Transfer address record (optional)

Offset	Size	Contents
0	1	`$16`, transfer address indicator
1–2	2	Entry point, big-endian

If a file has several transfer records (from appending binaries), the **last** one wins. When reading a binary file through the FMS, the calling program parses these records itself.

8.2 Text Files

Printable ASCII plus a few control characters:

Byte	Meaning
`$0D`	End of line or record
`$00`	Ignored on read (not passed to the caller)

Byte	Meaning
`$18`	Ignored on read (not passed to the caller)
`$09`	Space compression: the next byte is a count (2–127) of spaces

Space compression is transparent when the FCB has the file open in text mode. With the FCB Space Compression Flag (FCB byte 59) set for binary access (`$FF`), the `$09` and count bytes come through raw. A tool reading text files straight off a disk image has to expand `$09 <count>` itself and drop `$00` and `$18`.

---

9. Random Files and the File Sector Map

A file whose directory entry has a non-zero FSM flag (byte 19) is a random file. Its first two sectors, record 0, are the **File Sector Map**, which lets the FMS position to any record without walking the chain. The FSM is always exactly two sectors, so a random file with 5 data sectors occupies 7. When opened for sequential read, the FMS skips the FSM automatically.

A random file is created by opening for write and setting the FCB's FSM byte (FCB byte 23) to non-zero immediately after the open. Only text-type files make sense in random mode. Random files keep their random status when copied.

If the file is so fragmented that its extents don't fit in two sectors, the FMS returns error 23, `SECTOR MAP OVERFLOW – DISK TOO SEGMENTED`.

> **⚠ Not in the TSC guides**
>
> The internal layout of the FSM isn't documented in either guide. The commonly documented layout, which should be checked against real disk images, is: each FSM sector keeps the standard 4-byte header, and its 252 data bytes hold 3-byte extent entries (start track, start sector, count of contiguous sectors), 84 per sector, terminated by a zero entry.

---

10. Formatting a Disk (NEWDISK Procedure)

How a valid FLEX disk is built from blank media:

1. **Format all tracks** with 256-byte sectors, and write each sector's bytes 0–1 to link to the next logical sector. The last sector of each track links to sector 1 of the next track. Apply

interleave here if wanted.

2. **Verify every sector.** A bad sector on track 0 at sector 5 or below is fatal. Elsewhere, link bad sectors out of the chain and decrement the data sector count.
3. **Terminate the free chain** by zeroing the link of the last good sector on the disk.
4. **Initialize the directory:** zero the link of the last good sector on track 0, and zero the data bytes of every directory sector.
5. **Write the SIR** (§5).
6. **Write the boot loader** to track 0 sector 1 (and sector 2 if needed).

---

11. Limits

- Track and sector numbers are 8 bits, so addressing is at most 256 tracks × 255 sectors.
- Volume and file sizes in sectors are 16 bits, at most 65,535 sectors (about 16 MB of payload).
- Dates store a two-digit year.

Related

- UniFLEX
- Source PDFs: `6809fadg.pdf` (Adaptation Guide), FLEX Advanced Programmer's Guide (OCR'd with `ocrmypdf`)